

Modeling Stock Market Prediction Using Historical Data With Neural Networks

Grant Willis

Jack DiTallo

Gleb Usov

Varun Das

Brianna Persinger

Phillip Roberts

Yash Agarwal

University of Cincinnati, Experiential Learning AI Training Program

Nick Jarvis, Advisor

23 July 2025

Abstract

Several approaches for analyzing the stock market have been used including fundamental and technical analysis. Our project focuses on designing an AI model to effectively predict the closing price of a certain stock, only using the market's historical data. We experimented with the **Time-Series-To-Vector framework (TS2Vec)** for identifying technical patterns from unlabeled data, though this was not implemented into our models. Instead, our primary work was a comparison between the use of a **Convolutional Neural Network (CNN)** and a **Long-Short-Term-Memory (LSTM)** model. Training for both models was done using stock for Apple Inc. for a ten year period. From our results, both models were able to generalize the trend of the actual close price, but with a lag during any significant increases or decreases in the stock price, which is suspected to be due to the lack of market sentiment or other external factors in the dataset. In examining the CNN model, there was also an inexplicable spike during a rather small amount of price movement in the period examined. With sentiment in mind, we also experimented with data scraping, but found we were unable to effectively implement our findings in time, not only for developing a system, but also the potential amount of computing time to effectively retrieve a certain sentiment score for each day in a ten year period.

Introduction

The stock market is a place where investors can buy or sell shares of publicly traded companies ("Stock Market-Economy Relationship," 2025). In the US, the S&P 500, an index - group - composed of 500 large companies ("Dow Jones vs. S&P 500", 2025) that operate at a global scale and are traded in US markets, can be viewed as a representation of the US equity market. Another major index for measuring overall economic standing is the Dow Jones Industrial Average (DJIA). In fact, the exchange markets have such an influence on economic wellness that today's market regulations are hugely a result of a near-total economic crash that

happened October 19th, 1987. This event was later referred to as the infamous 'Black Monday Crash of 1987'. On this day, the DJIA saw a 22% loss of value (E., 2024), the largest single-day dip in history. Many factors influence the market including historical pricing data, previous trends, and public opinion on the value of a stock. The biggest factor leading to this market crash was determined to be automated trading programs. That day when the programs started selling, people panicked and also sold. Many lost money that day not just in America, but across the world. This is one instance where using automated programs did not have a good outcome, but it hasn't completely discouraged the use of programs for stock exchange. Our group designed an AI model whose goal is to predict a stock's closing price using technical analysis so its user can make reliable trade decisions. We focused our efforts primarily on analysis of technical indicators, which are 'mathematical calculations on certain parts of a stock's performance' (Bohen, 2021).

Experimental work with TS2Vec — Core Idea and Mechanics

While not used with our models, we also experimented on unsupervised learning with a method called TS2Vec ("Time-Series-to-Vector"). This was because this model could have the potential to improve an AI model's recognition of patterns. TS2Vec is an unsupervised framework that converts any multivariable sequential data into compact, machine-interpretable attributes at every time step. The process in which this data is trained is enforced by a *hierarchical contrastive loss function*. Essentially, for each time step, different versions of it are created but with slight changes. This is done to be able to describe the same time step in different ways. These versions are represented in a point in space. This ultimate goal of the model is to keep these versions of the same time series together, while keeping different time steps, and the many versions that are derived from them, apart (Yue et al., 2021).

By applying this objective at several *sub-sampling depths* (e.g., every 1, 2, 4, 8 ... steps), the encoder becomes sensitive to both short-term wiggles and long-range structure. Essentially, TS2Vec uses a stack of **convolutions** and is able to skip some of these layers to aid in having the information flow through. This results in a training model that is not only able to retain fine-grained features, but also self-supervised, where there is no need for labels such as with classification. Instead, the necessary aspects needed are the sequences alone, resulting in a cluster, or grouping of data points (Yue et al., 2021).

We implemented this model by using the last three years as the test set, more specifically from the beginning of 2022 to the same day in 2025 were clustered. For the train set, we utilized the previous year's data, from 2021 to 2022.. The set was divided into 256 day windows on each close price. To normalize the data, each window starts at zero so as to be able cluster each window regardless of the price. As for the convolution, the stride was equal to one. From these normalized windows, with each being represented by a 512 dimensional vector, each was clustered by k-means into eight separate clusters.

Results

Figure 1 shows AAPL's test-period Close price (gray) overlaid with 189 colored markers, one per 256-day window, indicating its cluster. Markers now scatter across all price levels, confirming clusters capture *shape*, not absolute level.

Figure 1

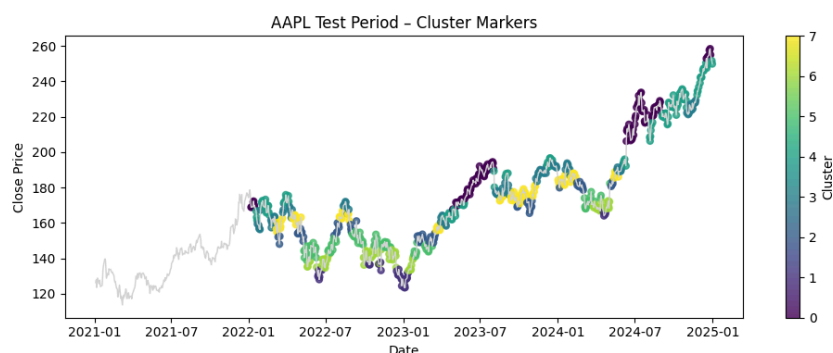
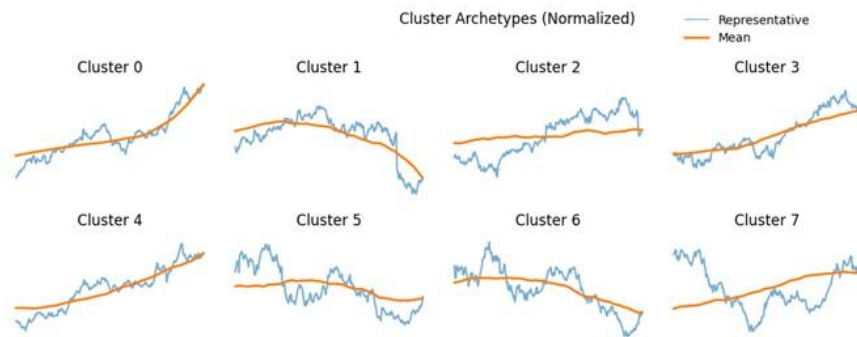


Figure 2

For each cluster in **Figure 2**, we were able to find several patterns that would apply throughout. The blue line represents a single window of the test



set that aligns closest to the data, with the orange line being the average of all of the windows associated with the cluster. While we were able to successfully classify the data, we have yet implemented a solution that would aid in predicting the close price. With further study, this method of clustering data would be used to aid other AI models in detecting patterns, potentially leading to a greater amount of accuracy.

CNN Modeling Approach

One of our AI models that did implement stock price prediction begins by importing historical stock price data from a file using pandas, then filters for a specified company to predict upon (The pandas development team, 2025). The data taken is filtered into five main aspects, Open, High, Low, Close, and Volume and stored in chronological order for when it is to be trained. Then StandardizedScalar, a tool from sklearn, normalizes the data and segments into 15 day windows that are overlapped over each other to help the model learn temporal patterns (Pedregosa et al., 2011). The predicted value for each corresponding window is the closing price on the immediately following day which is also scaled using the same standardization process. The dataset with the five main aspects of the data types is then separated into an unshuffled train test split also using sklearn (Pedregosa et al., 2011). The model uses PyTorch to utilize one-dimensional convolutional layers to scan over the time series and detect important

short-term trends across the five input features: open, high, low, close, and volume, as well as to structure the data throughout several layers. The most important patterns are what is focused on by max pooling to reduce the data size. After the convolutional layers, the data is flattened and passed through fully connected layers that help the model learn deeper relationships. The final output layer predicts the closing price on the following day. LeakyReLU activation is what the model utilizes to introduce more complex patterns and Mean Squared Error measures the loss compared to the actual prices for our model (Paszke et al., 2019).

LSTM Modeling:

For modeling time series data for stock price prediction, we also considered using a Recurrent Neural Network (RNN). Unlike standard, feedforward, neural networks, an RNN is able to observe a number of data points over time, storing the information in the “hidden state.” This state is the combination of previous inputs. At each time step, the hidden state is changed based on the current input as well as the previous hidden state (Elman, 1990).

The hidden state would then be updated with the sum of a new input and the previous hidden state, each being multiplied by a weight. Then, an activation function would be used, hyperbolic tangent in this case, to contain the result between negative one and one. This would aid in explaining the relationship of how one point of data would change the next, due to previous inputs being able to affect the results of the next. This method can then be repeated for several points, creating a phenomenon where the neural network can find patterns between a large number of points (Williams & Zipser, 1989).

A significant problem with this model is related to the many gradients that are produced from each input. Using back propagation, a gradient, which is derived by a loss function, is found from how all previous time steps influence the current one. Yet, these results being weighted and able to have values greater or less than one, each consecutive result may

exponentially increase (explode) or decrease (vanish) this gradient. As such, the AI model would have a hard time finding patterns over several time steps (Hochreiter & Schmidhuber, 1997). This would significantly affect the efficacy of the neural network, as any large intervals of data being observed would completely ruin any reasonable conclusions to how one data point affects the next.

An LSTM (Long Short-Term Memory) neural network is designed to solve this problem, being able to observe over one thousand time steps (Hochreiter et al., 1997). This is done by including both long and short term memory values in determining the value of the hidden state. With any input entered in the model, the value goes through three different gates: input, output, and forget gates, with the forget gate being introduced in Gers et al., 1999. The forget gate is tasked at resetting any obsolete information in long term memory, the input gate determines how to update the long term memory, and the output gate updates the value of the short term memory. These updated long and short term values are then used as the initial memory used with the next input in the sequence (Gers et al., 1999).

LSTM Implementation:

In implementing the LSTM model, we decided to use stock data from Apple using yfinance to generate the historical data (Aroussi, n.d.). The model was trained on stock data that spanned for the last ten years in single day intervals. For each day, there was an open, high, low, close price, as well as the amount of volume. There was also an additional step of using pandas to save the results into a CSV file for offline use, without needing to reuse the yfinance API (The pandas development team, 2025; Aroussi, n.d.). This method would also aid in replicability with the model. Additional days in the data may slightly impede direct comparison with older models.

Numpy was then used for organizing the data for use with PyTorch and Matplotlib for plotting and visualization (Harris et al., 2020; Paszke et al., 2019; Hunter, 2007). Sklearn was

used to normalize the train set, while having the test set fit the scaling in order to prevent data leaks (Pedregosa et al., 2011).

The model was created using Pytorch, utilizing a single LSTM layer with a hidden size of 128 and trained with 32 epochs (Paszke et al., 2019). The model examined the data in window sizes of 60 days. The data was split into eighty percent training, twenty percent test. Pytorch which was used for organizing the data into an applicable format for training, utilizing the DataLoader for processing the data into 32 batches and the AdamW optimizer was for training the data (Paszke et al., 2019).

With this information, the LSTM model is tasked with predicting the close price on a particular day from the data of the previous sixty days. It is worth noting that the first sixty days when the market is open would not be represented on the graph, as those would be used in predicting the close price of the sixty first day. Along with the number of trading days not aligning with the number of days in a year, would make the predictions be from a twenty one month time span, starting on 9 October 2023 and ending on 15 July 2025.

Comparison of LSTM and CNN AI Models

Figure one shows the comparison between the two models. The reason for the gap between the start dates for both models lies in the window sizes. The LSTM model is examining windows in sixty days, while the CNN model is examining fifteen. Both models seem to be



able to predict the general trends of the stock data as seen in figure one. Both also have similar, but small amounts of loss using the Mean Squared Error method.

However, it seems that the two approaches differ to a greater extent from a more local perspective. In figure two, it seems that the LSTM model more accurately predicts the trends of

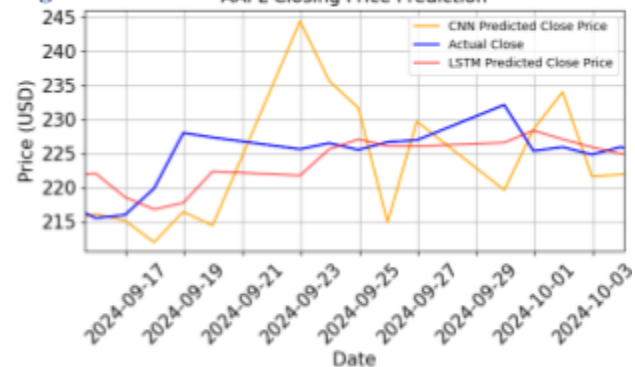
the data. The CNN model seems more likely to display rapid ups and downs, while the LSTM model provides a smoother gradation in comparison. These rapid changes in price prediction may have been the reason for the inaccurately large spike, which peaked at 23 September 2024, as seen more closely in figure three. This is the result of the more step by step predictive approach in LSTM as well as the greater window size (of sixty days compared to only fifteen), making the model less sensitive to local values. The more amount of data that the model can

observe, the less significant each data point would be in predictions. Due to its' smaller window size, the CNN places a greater emphasis on localized patterns, potentially being more sensitive to them. As such, there would be more variability between points. Figure two also shows that the models seem to struggle with responding to sudden spikes and dips in the stock price. Here, the models seem to respond to these changes after the changes take place. A reason for this can be from the dataset only including data from stock prices alone. Without company revenue or market sentiment, aspects that can greatly affect investor demand in a certain stock, there would be less predictive power behind the decisions that the model makes.

Figure 2: AAPL Closing Price Prediction



Figure 3: AAPL Closing Price Prediction



CNN MSE Loss: 0.0064 LSTM MSE Loss: 0.0072

Conclusion

Our study was able to get promising results using our different modeling approaches. The CNN model proved to be able to track a general trend of stock prices, but when tasked to track day to day, it struggled a bit more compared to the LSTM model due to having a greater variance in its predictions. Similarly, the LSTM model was also able to track general trends fairly well, but struggled slightly with abrupt changes in stocks as it tended to lag behind the actual close price. The model also consistently underpredicted the actual values. Overall, the two models created worked fairly well by using historical stock pricing data to predict a general accurate trend in future prices. A limiting factor with our models is that they are unable to account for outside factors that influence stock prices. For example, for tech companies like Apple, our models can't account for new product releases that may lead to a spike in stock price, or the general sentiment of the company. By integrating an awareness of events that affect stock price and consumer/investor outlook, the changes in the stock market can be modeled more accurately due to outside factors being evaluated as well. We would like to implement a natural language processing approach along with the models we have developed to help account for outside factors that could be captured by analyzing sentiment about companies, the stock market, and the economy as a whole. By using this information, there will be a much more diverse perspective for the AI to look at to create even more accurate predictions. We are leaving these ideas for future exploration. Ultimately, the applied use of an AI model to predict changing stock prices has proven to be a success and can be expanded on in many more directions.

References

- Aroussi, R. (n.d.). *yfinance: Download market data from Yahoo! Finance's API*. GitHub.
<https://github.com/ranaroussi/yfinance>
- Bohen, T. (2021, December 29). The Complete Guide to Technical Indicators {With Infographic}. StocksToTrade; StocksToTrade.
<https://stockstotrade.com/types-technical-indicators-infographic/>
- Chen, J. (2006, October 29). *Technical Indicator: Definition, Analyst Uses, Types and Examples*. Investopedia; Investopedia.
<https://www.investopedia.com/terms/t/technicalindicator.asp>
- Dow Jones vs. S&P 500: Key Differences in Stock Market Indices*. (2025, July 21). AI Tools for Stock Forecasts & Price Predictions | Tickeron; Tickeron.
<https://tickeron.com/trading-investing-101/what-are-the-key-distinctions-between-the-dow-jones-industrial-average-and-the-s-p-500/>
- Elman, J.L. (1990), Finding Structure in Time. *Cognitive Science*, 14: 179-211.
https://doi.org/10.1207/s15516709cog1402_1
- E., V. (2024, January 4). *Black Monday Crash of 1987: Insights & Impact | MarketBulls*. MarketBulls. <https://market-bulls.com/black-monday-crash-of-1987/>
- F. A. Gers, J. Schmidhuber and F. Cummins, "Learning to forget: continual prediction with LSTM," 1999 Ninth International Conference on Artificial Neural Networks ICANN 99. (Conf. Publ. No. 470), Edinburgh, UK, 1999, pp. 850-855 vol.2, doi: 10.1049/cp:19991218.
- Harris, C.R., Millman, K.J., van der Walt, S.J. et al. *Array programming with NumPy*. *Nature* 585, 357–362 (2020). DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2). ([Publisher link](#)).
- Hunter, J. D. (2007). *Matplotlib: A 2D graphics environment*. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>

The pandas development team. (2025). pandas-dev/pandas: Pandas (v2.3.1). Zenodo.

<https://doi.org/10.5281/zenodo.15831829>

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019).

PyTorch: An Imperative Style, High-Performance Deep Learning Library. In

Advances in Neural Information Processing Systems 32 (pp. 8024–8035). Curran

Associates, Inc. Retrieved from

<http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M.,

Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau,

D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). *Scikit-learn: Machine*

learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

<https://www.jmlr.org/papers/v12/pedregosa11a.html>

Ronald J. Williams, David Zipser; A Learning Algorithm for Continually Running Fully

Recurrent Neural Networks. *Neural Comput* 1989; 1 (2): 270–280. doi:

<https://doi.org/10.1162/neco.1989.1.2.270>

S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," in *Neural Computation*, vol. 9,

no. 8, pp. 1735-1780, 15 Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

What's the relationship between the stock market and the economy? (2025). RBC Global Asset

Management.

<https://www.rbcgam.com/en/ca/learn-plan/investment-basics/whats-the-relationship-between-the-stock-market-and-the-economy/detail>

Yue, Z., Zhang, H., Wang, S., Yan, J., & Sun, L. (2021). *TS2Vec: Towards universal*

representation of time series [Preprint]. arXiv. <https://arxiv.org/abs/2106.10466>